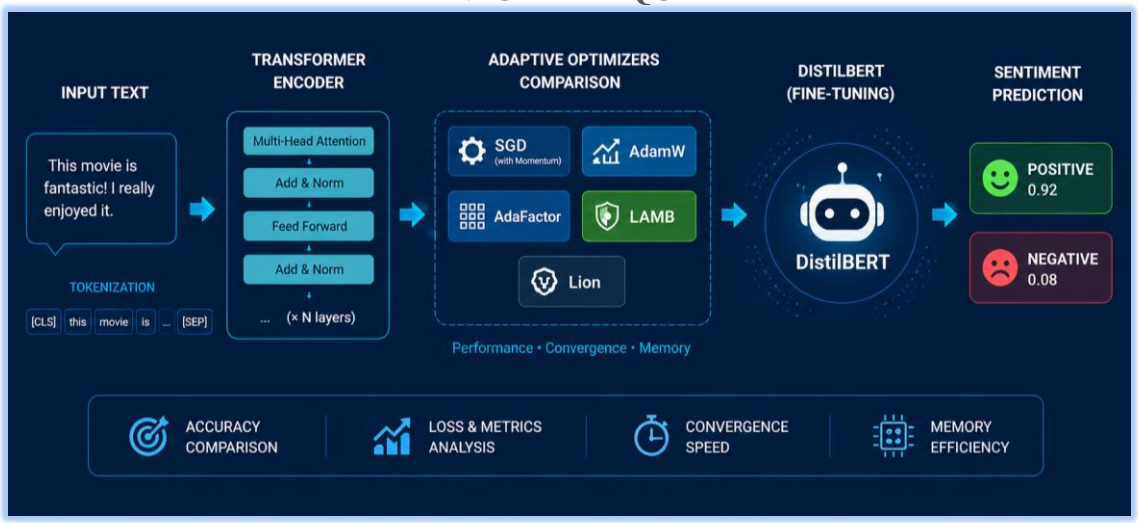
UNIVERSITÉ HASSAN II DE CASABLANCA
FACULTÉ DES SCIENCES BEN M'SICK

Master M1 Big Data & Data Sciences

DÉPARTEMENT DE MATHÉMATIQUES ET INFORMATIQUE



FINAL PROJECT REPORT

Optimization Algorithms Module

Study of Adaptive Gradient Methods for Transformer Fine-Tuning in NLP

A Comparative Analysis of AdamW, AdaFactor, LAMB and Lion on IMDB Sentiment Classification using DistilBERT

Prepared by:

- GHAZLI MOHAMED AMINE
- BOULATAR MOUAD
- HAMROUDI AYOUB
- MOUSLIM JAD

Supervisor :

Faouzia Benabbou

Academic Year 2025–2026

Abstract

This academic research project presents a thorough comparative study of five gradient descent optimization algorithms for fine-tuning a lightweight Transformer model on a Natural Language Processing (NLP) task. We compare a classical non-adaptive baseline (SGD with Momentum) against four modern adaptive gradient optimizers: AdamW, AdaFactor, LAMB, and Lion. The evaluation is carried out on the DistilBERT architecture for binary sentiment classification using the IMDB movie reviews dataset. The optimizers are analyzed along several critical performance axes: convergence speed, training stability, learning rate sensitivity, batch size stability, GPU peak memory consumption, and layer-wise gradient norm dynamics. Empirical results demonstrate that AdaFactor achieves the best overall trade-off, securing the highest test F1-score (89.72%) while reducing peak GPU memory usage by approximately 30% compared to standard AdamW. Conversely, the non-adaptive SGD baseline fails to converge efficiently within the allotted epochs, highlighting the necessity of parameter-wise adaptive learning rates for optimizing Transformer models. A transferability extension template using the MiniLM architecture is also provided to support further research.

Keywords: Adaptive optimization, Transformers, Deep Learning, NLP, DistilBERT, AdaFactor, AdamW, Lion, Sentiment analysis.

Contents

List of Figures

List of Tables

1. Introduction

1.1 Context and Problem Statement

In the contemporary landscape of Natural Language Processing (NLP), text classification is a cornerstone application, enabling systems to categorize text corpus, extract sentiment, and analyze user behavior. Sentiment analysis on long narratives, such as movie reviews, presents specific challenges since text reviews often contain subtle linguistic cues, negations, and complex dependency structures. Performing this classification with high accuracy requires deep neural networks capable of learning hierarchical semantic representations. However, training these models or fine-tuning them on specific target domains is computationally expensive and highly sensitive to optimization trajectories.

1.2 Importance of Transformer Architectures in NLP

Since their introduction by Vaswani et al. (2017), Transformer architectures based entirely on self-attention mechanisms have revolutionized NLP, replacing recurrent networks (RNNs) and convolutional networks (CNNs). Models like BERT (Bidirectional Encoder Representations from Transformers) by Devlin et al. (2018) pre-train bidirectionally on vast text corpora, capturing rich contextual semantics. While BERT achieves remarkable generalizability, its high parameter count (~110 million for BERT-base) demands significant GPU memory and training time. To address this efficiency bottleneck, Sanh et al. (2019) introduced DistilBERT, a compressed variant trained via knowledge distillation. DistilBERT reduces the parameter count by 40% while preserving 97% of BERT's language understanding capabilities, making it an ideal model for academic benchmarks and resource-constrained deployment.

1.3 Role and Challenges of Optimization Algorithms

The optimization landscape of Transformer networks is highly non-convex, characterized by sharp curvatures, flat basins, and numerous saddle points. Traditional non-adaptive optimization methods, such as Stochastic Gradient Descent (SGD), apply a uniform learning rate to all parameters. While SGD works well for vision models, it struggles in Transformers because different layers (such as the embedding matrix vs. the attention heads vs. the classification head) experience drastically different gradient scales. To overcome this, adaptive optimization methods (e.g., AdamW, AdaFactor, Lion) dynamically scale the learning rate for each individual parameter based on historical gradient moments. Decoupling weight decay from the gradient updates (as in AdamW) further stabilizes regularization, making adaptive optimizers the default choice.

1.4 Scientific Objectives of the Benchmark

The primary goal of this research is to carry out a rigorous empirical study comparing five optimization methods for fine-tuning DistilBERT on the IMDB sentiment classification task. We compare a non-adaptive baseline (SGD with Momentum) against four modern adaptive methods: AdamW, AdaFactor, LAMB, and Lion. Our objectives are to analyze their convergence speed, learning rate sensitivity, batch size robustness, memory efficiency, and internal layer-wise gradient dynamics. This multi-dimensional evaluation will help identify the optimal trade-off between task performance and resource efficiency.

2. Literature Review and Theoretical Foundations

2.1 Related Work

Optimization in deep learning has evolved from simple first-order descent algorithms to complex adaptive schemes. The standard Adam optimizer, proposed by Kingma & Ba (2014), combined the concepts of momentum and adaptive learning rates, quickly becoming the standard for deep neural networks. However, Loshchilov & Hutter (2017) demonstrated that the traditional L2 regularization implementation in Adam leads to suboptimal generalization because the weight decay is scaled by the inverse of the gradient variance. They proposed AdamW, which decouples the weight decay update, yielding substantial generalization gains in NLP tasks.

To mitigate the high GPU memory overhead of storing running moments for millions of parameters, Shazeer & Stern (2018) introduced AdaFactor. By factorizing the second-moment accumulator matrix, AdaFactor achieves sublinear memory complexity. For large-scale distributed training, You et al. (2019) introduced LAMB, which applies layer-wise normalization (trust ratios) to prevent exploding updates, enabling training with very large batch sizes. More recently, Chen et al. (2023) proposed Lion, which uses an evolutionary-derived algorithm that utilizes only the sign of the momentum, reducing memory use (storing only one moment state instead of two) and regularizing parameter updates.

2.2 Comparative Literature Summary Table

Table 2.1: Comparative literature summary on Transformer optimizers.

Author	Optimizer	Target Domain	Key Conclusion
Loshchilov & Hutter (2017)	AdamW	General Deep Learning & NLP	Decouples weight decay from adaptive updates, significantly improving generalization in Transformers.
Shazeer & Stern (2018)	AdaFactor	Large NLP Models (T5)	Factorizes the second-moment accumulator to reduce GPU memory footprint while maintaining convergence properties.
You et al. (2019)	LAMB	Large-Batch Training (BERT)	Introduces layer-wise trust ratios, stabilizing training and allowing scaling up of batch sizes to 32k.

Chen et al. (2023)	Lion	Vision & Language Modeling	Discovers a sign-based momentum optimizer via program search, requiring less memory and accelerating computation.

2.3 Theoretical Foundations of Optimization Algorithms

2.3.1 Stochastic Gradient Descent with Momentum (SGD + Momentum)

Principle and Intuition: Stochastic Gradient Descent (SGD) with Momentum updates the model parameters along the negative direction of the accumulated velocity. This approach introduces physical-like momentum that accelerates learning in direction coordinates of consistent gradient sign, while dampening high-frequency oscillations. The mathematical updates are formulated as follows:

$$v_t = \gamma v_{t-1} + \eta(\nabla f(\theta_t) + \lambda \theta_t) \quad \textbf{(Eq. 2.1)}$$

- v_t : velocity vector at the current time step t
- v_{t-1} : accumulated velocity vector from the previous step t-1
- γ : momentum decay coefficient (damping factor, typically set to 0.9)
- η : global learning rate parameter
- $\nabla f(\theta_t)$: first-order gradient of the loss function with respect to the parameters at step t
- θ_t : model parameter vector at step t
- λ : L2 regularization (weight decay) coefficient

Equation (2.1) computes the velocity vector by accumulating the exponentially damped past updates with the current regularized gradient. The momentum coefficient γ dampens high-frequency noise and stabilizes the search direction, while the weight decay term λ prevents parameter explosion by penalizing large weights directly inside the velocity step.

$$\theta_{t+1} = \theta_t - v_t \quad \textbf{(Eq. 2.2)}$$

- θ_{t+1} : updated parameter vector for the next step t+1
- θ_t : current parameter vector at step t
- v_t : computed velocity vector at step t

Equation (2.2) updates the parameter vector by subtracting the computed velocity vector, shifting the model parameters along the accumulated momentum trajectory. By updating the

weights using this smoothed velocity rather than the instantaneous gradient, the optimizer avoids oscillations in steep ravines and easily rolls over flat regions or local minima.

2.3.2 AdamW (Decoupled Weight Decay)

Principle and Intuition: AdamW is an extension of Adam that decouples weight decay from the adaptive gradient updates. By tracking both the first and second moments of the gradients, it adjusts the step size coordinate-wise, which stabilizes training on complex topologies. Decoupling weight decay prevents L2 regularization from being scaled by the historical gradient variance, restoring the effectiveness of regularizing the model parameters. The update sequence is formulated below:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad \textbf{(Eq. 2.3)}$$

- m_t : first moment vector (running average of gradients) at step t
- m_{t-1} : first moment vector from the previous step t-1
- β_1 : exponential decay rate for the first moment (typically 0.9)
- g_t : current gradient vector at step t, where $g_t = \nabla f(\theta_t)$

Equation (2.3) computes the exponentially weighted moving average of past gradients. The parameter β_1 controls the influence of historical gradients and introduces momentum into the optimization process, thereby reducing oscillations and accelerating convergence in relevant directions.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \textbf{(Eq. 2.4)}$$

- v_t : second moment vector (running average of squared gradients) at step t
- v_{t-1} : second moment vector from the previous step t-1
- β_2 : exponential decay rate for the second moment (typically 0.999)
- g_t^2 : element-wise squared gradient vector

Equation (2.4) computes the running average of the squared gradients, capturing the uncentered variance along each coordinate direction. The decay parameter β_2 determines the tracking memory, which dynamically scales down updates in high-variance directions and accelerates updates in low-variance coordinates.

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \textbf{(Eq. 2.5)}$$

- $\hat{m}_t$: bias-corrected first moment estimate at step t
- $\hat{v}_t$: bias-corrected second moment estimate at step t
- m_t : uncorrected first moment vector at step t
- v_t : uncorrected second moment vector at step t

- β_1^t : first-moment decay rate raised to the power of the current step t
- β_2^t : second-moment decay rate raised to the power of the current step t

Equation (2.5) applies bias correction to the first and second moment accumulators to correct for their initialization at zero. This scaling stabilizes the update step sizes during the early iterations of training, preventing the model from experiencing large, unstable updates when the step count t is small.

$$\theta_{t+1} = \theta_t(1 - \eta\lambda) - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t \qquad \textbf{(Eq. 2.6)}$$

- θ_{t+1} : updated parameter vector at the next step t+1
- θ_t : current parameter vector at step t
- η : global learning rate
- λ : decoupled weight decay coefficient
- $\hat{m}_t$: bias-corrected first moment estimate
- $\hat{v}_t$: bias-corrected second moment estimate
- ε : small stabilizing constant (typically 1e-8) to prevent division by zero

Equation (2.6) updates the parameters by applying decoupled weight decay directly to the weights, followed by the adaptive gradient step. This decoupling prevents the L2 regularization step from being scaled down by the second-moment accumulator $\hat{v}_t$, maintaining a consistent regularization strength across all parameters.

2.3.3 AdaFactor (Sublinear Memory Cost)

Principle and Intuition: AdaFactor is designed to reduce the memory footprint of storing second-moment estimates during large-scale Transformer training. While AdamW requires storing a second-moment tensor of size equivalent to the model parameters, AdaFactor factorizes the second-moment accumulator matrix along the row and column dimensions. This factorization reduces memory complexity from quadratic to sublinear, while maintaining comparable convergence properties. The update steps are formulated as follows:

$$R_{t,i} = \beta_2 R_{t-1,i} + (1 - \beta_2)\sum_j g_{t,i,j}^2 \qquad \textbf{(Eq. 2.7)}$$

- $R_{t,i}$: row-wise second-moment accumulator vector at index i and step t
- $R_{t-1,i}$: row-wise second-moment accumulator vector from step t-1
- β_2 : exponential decay rate for the second-moment estimate
- $g_{t,i,j}^2$: squared gradient for the parameter at row i and column j

Equation (2.7) computes the row-wise running average of the squared gradients for a 2D parameter matrix. By accumulating variance along the rows rather than tracking each parameter coordinate independently, AdaFactor reduces the memory requirement of the second moment from quadratic to sublinear.

$$C_{t,j} = \beta_2 C_{t-1,j} + (1-\beta_2)\sum_i g_{t,i,j}^2 \quad \textbf{(Eq. 2.8)}$$

- $C_{t,j}$ **:** column-wise second-moment accumulator vector at index j and step t
- $C_{t-1,j}$ **:** column-wise second-moment accumulator vector from step t-1
- β_2 **:** exponential decay rate for the second-moment estimate
- $g_{t,i,j}^2$ **:** squared gradient for the parameter at row i and column j

Equation (2.8) computes the column-wise running average of the squared gradients, storing a 1D vector of size equal to the column dimension. Together with the row accumulator, this factorization reduces the second-moment tracking memory footprint from O(R*C) to O(R+C), yielding substantial memory savings.

$$\hat{V}_{t,i,j} = \frac{R_{t,i} C_{t,j}}{\sum_k R_{t,k}} \quad \textbf{(Eq. 2.9)}$$

- $\hat{V}_{t,i,j}$ **:** reconstructed factorized second-moment estimate for cell (i, j) at step t
- $R_{t,i}$ **:** row accumulator value at row index i
- $C_{t,j}$ **:** column accumulator value at column index j
- $\sum_k R_{t,k}$ **:** sum of all elements in the row accumulator vector

Equation (2.9) reconstructs the full second-moment matrix for each parameter coordinate by computing the normalized outer product of the row and column vectors. This rank-1 factorization serves as an efficient approximation of the gradient variance, enabling parameter-wise learning rate adaptation with negligible memory overhead.

$$U_t = \frac{g_t}{\sqrt{\hat{V}_t} + \epsilon} \quad \textbf{(Eq. 2.10)}$$

- U_t **:** normalized update matrix at step t
- g_t **:** current gradient matrix at step t
- $\hat{V}_t$ **:** reconstructed second-moment matrix
- ε **:** stabilizing parameter to prevent division by zero (typically set to 1e-30)

Equation (2.10) normalizes the current gradient matrix element-wise using the reconstructed second-moment estimate. By scaling down updates in directions of high gradient variance, it ensures stable update steps and prevents unstable updates on parameters with noisy gradients.

$$\boldsymbol{\theta_{t+1} = \theta_t(1 - \eta\lambda) - \eta U_t} \quad \textbf{(Eq. 2.11)}$$

- θ_{t+1} **:** updated parameter matrix for step t+1
- θ_t **:** current parameter matrix at step t
- η **:** learning rate parameter
- λ **:** decoupled weight decay coefficient
- U_t **:** normalized update matrix

Equation (2.11) applies the parameter update by subtracting the normalized gradient step while applying decoupled weight decay. This update rule enables stable convergence comparable to AdamW while reducing peak memory requirements, facilitating the fine-tuning of large Transformer models under GPU limits.

2.3.4 LAMB (Layer-wise Adaptive Moments for Large Batch)

Principle and Intuition: LAMB is designed to stabilize large-batch training of deep Transformer architectures. By computing trust ratios that compare the L2 norm of the layer parameters to the L2 norm of the adaptive updates, it normalizes step sizes layer-by-layer. This prevents gradient explosion or divergence, even when using extremely large batch sizes. The updates are formulated as follows:

$$\boldsymbol{u_t^l = \frac{\hat{m}_t^l}{\sqrt{\hat{v}_t^l + \epsilon}} + \lambda\theta_t^l} \quad \textbf{(Eq. 2.12)}$$

- u_t^l **:** base update vector for layer l at step t
- $\hat{m}_t^l$ **:** bias-corrected first-moment estimate for layer l
- $\hat{v}_t^l$ **:** bias-corrected second-moment estimate for layer l
- ε **:** small stabilizer to prevent division by zero
- λ **:** weight decay coefficient
- θ_t^l **:** parameter vector for layer l at step t

Equation (2.12) calculates the base adaptive update vector for layer l using Adam-style moments combined with decoupled weight decay. This unscaled vector represents the raw update step for the layer's parameters, which is later normalized to match the layer-wise weight scale.

$$\boldsymbol{\phi_t^l = \frac{\|\theta_t^l\|_2}{\|u_t^l\|_2}} \quad \textbf{(Eq. 2.13)}$$

- ϕ_t^l : layer-wise trust ratio scaling factor for layer l at step t
- $\|\theta_t^l\|_2$: L2 norm of the parameter vector at layer l
- $\|u_t^l\|_2$: L2 norm of the proposed base update vector u_t^l

Equation (2.13) computes the layer-wise trust ratio by comparing the L2 norm of the layer's weights to the norm of the proposed updates. This ratio stabilizes the optimization path by preventing the update steps from dominating the weight scales, particularly during large-batch training.

$$\theta_{t+1}^l = \theta_t^l - \eta \phi_t^l u_t^l \quad \textbf{(Eq. 2.14)}$$

- θ_{t+1}^l : updated parameter vector for layer l at step t+1
- θ_t^l : current parameter vector for layer l at step t
- η : global learning rate
- ϕ_t^l : trust ratio for layer l
- u_t^l : base update vector for layer l

Equation (2.14) updates the parameters of layer l by scaling the adaptive update vector by the layer's trust ratio ϕ_t^l. This layer-wise step matching maintains stability and prevents gradient explosion or divergence, even when training with large batch sizes.

2.3.5 Lion (EvoLved Sign Momentum)

Principle and Intuition: Lion is a memory-efficient, sign-based optimization algorithm discovered through program search. Instead of scaling updates based on historical gradient variance (second moments), Lion updates parameters using only the sign of the gradient momentum. This results in uniform update step sizes across all coordinates, which simplifies computation and provides a regularizing effect. The updates are formulated as follows:

$$c_t = \text{sign}(\beta_1 m_{t-1} + (1 - \beta_1) g_t) \quad \textbf{(Eq. 2.15)}$$

- c_t : sign-based update direction vector at step t
- m_{t-1} : momentum history vector from the previous step t-1
- β_1 : exponential decay factor for momentum in the update step (typically 0.9)
- g_t : current gradient vector at step t

Equation (2.15) computes the sign-based update direction from the convex combination of the current gradient and the momentum. Extracting only the sign normalizes the step size coordinate-wise, which adds a strong regularizing effect and dampens gradient noise.

$$\theta_{t+1} = \theta_t(1 - \eta\lambda) - \eta c_t \quad \textbf{(Eq. 2.16)}$$

- θ_{t+1} : updated parameter vector at the next step t+1

- θ_t : current parameter vector at step t
- η : global learning rate
- λ : decoupled weight decay coefficient
- c_t : sign-based update direction vector

Equation (2.16) applies the update by subtracting the sign direction scaled by the learning rate, alongside decoupled weight decay. Because every coordinate is updated by the exact same magnitude η, the optimization trajectory is highly uniform, accelerating training but increasing sensitivity to the learning rate.

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t \quad \textbf{(Eq. 2.17)}$$

- m_t : updated momentum history vector at step t
- m_{t-1} : previous momentum history vector at step t-1
- β_2 : exponential decay rate for momentum tracking (typically 0.99)
- g_t : current gradient vector at step t

Equation (2.17) updates the momentum history by tracking the running average of the gradients. By tracking only a single moment vector (instead of two as in AdamW), Lion reduces the memory footprint, saving GPU memory in resource-constrained training environments.

2.4 BERT and DistilBERT Language Modeling

Modern pre-trained language models like BERT (Bidirectional Encoder Representations from Transformers) are built on stacked Transformer encoder blocks. Each encoder block contains a multi-head self-attention layer followed by a position-wise feed-forward network, with residual connections and layer normalization. Because self-attention is permutation-invariant, positional information must be added to the input embeddings (using learned positional embeddings or sinusoidal encodings) to maintain sequence order. The self-attention mechanism maps a set of query, key, and value vectors derived from the input sequence embeddings. Mathematically, the scaled dot-product attention is formulated as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \quad \textbf{(Eq. 2.18)}$$

- **Q :** Query matrix of dimension N x d_k, representing token queries
- **K :** Key matrix of dimension N x d_k, representing token keys
- **V :** Value matrix of dimension N x d_v, representing token values
- d_k : dimensionality of the query and key vectors, used as a scaling factor
- **softmax :** row-wise softmax activation function

Equation (2.18) computes the scaled dot-product attention, allowing tokens to dynamically focus on relevant context in the sequence. The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large, which would cause the softmax gradients to vanish and stall training.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad \textbf{(Eq. 2.19)}$$

- **MultiHead(Q, K, V) :** final multi-head attention representation matrix
- **head_i :** output of the i-th individual attention head
- **h :** number of parallel attention heads (typically 12 in BERT-base)
- **W^O :** learned output linear projection matrix that merges the concatenated head outputs
- **Concat :** operator that concatenates matrices column-wise

Equation (2.19) combines the outputs of the parallel attention heads and projects them using the output weight matrix W^O. This multi-head structure allows the model to simultaneously attend to information from different representation subspaces, capturing complex syntactic and semantic relations.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad \textbf{(Eq. 2.20)}$$

- **head_i :** computed output vector for the i-th attention head
- **Q, K, V :** base query, key, and value representation matrices
- **W_i^{φ} :** learned linear projection matrix for queries in head i
- **$W_i^{И}$:** learned linear projection matrix for keys in head i
- **W_i^V :** learned linear projection matrix for values in head i

Equation (2.20) projects the queries, keys, and values of head i into specific low-dimensional subspaces. This enables each individual head to learn separate linguistic relationships (such as subject-verb agreement or coreference), enriching the model's contextual representations.

While BERT's bidirectional nature enables it to learn rich representations, fine-tuning its 110 million parameters is computationally expensive. DistilBERT addresses this by utilizing knowledge distillation during pre-training to compress the model. Knowledge distillation is a compression technique where a smaller 'student' network is trained to reproduce the behavior and output distribution of a larger 'teacher' network. The student model minimizes a multi-task loss function during training, which is formulated as follows:

$$L_{\text{total}} = \alpha L_{\text{MLM}} + \beta L_{\text{KD}} + \gamma L_c \quad \textbf{(Eq. 2.21)}$$

- **L_total :** total multi-task loss optimized during training
- **L_MLM :** standard masked language modeling loss

- **L_KD :** Kullback-Leibler divergence distillation loss
- **L_cosine :** cosine embedding loss between the student's and teacher's hidden states
- **α, β, γ :** scaling coefficients that balance the contribution of each loss term

Equation (2.21) combines standard masked language modeling loss with distillation and cosine alignment objectives. This multi-task loss forces the student model to mimic the teacher's output probabilities and hidden representations, preserving performance during compression.

$$L_{KD} = T^2 \cdot D_{KL}(\sigma(\frac{z_s}{T}) \parallel \sigma(\frac{z_t}{T})) \quad \textbf{(Eq. 2.22)}$$

- **L_KD :** Kullback-Leibler divergence distillation loss
- **T :** temperature parameter (typically T >= 1.0) used to soften probability distributions
- **D_KL :** Kullback-Leibler divergence operator that measures similarity between two distributions
- **σ :** softmax function
- **z_s :** raw logit outputs of the student network
- **z_t :** raw logit outputs of the teacher network

Equation (2.22) computes the divergence between the student's and teacher's softened probabilities, scaled by the temperature T. Softening the distributions reveals dark knowledge (secondary relationships between classes), allowing the student model to learn richer representations from the teacher.

3. Experimental Methodology

3.1 The IMDB Dataset and Exploration

The IMDB dataset is a standard benchmark for binary sentiment classification, consisting of 50,000 highly polarized movie reviews. For our experimental setup on Google Colab, we partitioned the dataset into a training subset, a validation subset, and a test subset. Figure 3.1 displays the distribution of review lengths (word counts) and shows the balanced binary label distribution (50% positive and 50% negative reviews), ensuring the dataset is free from class imbalance bias.

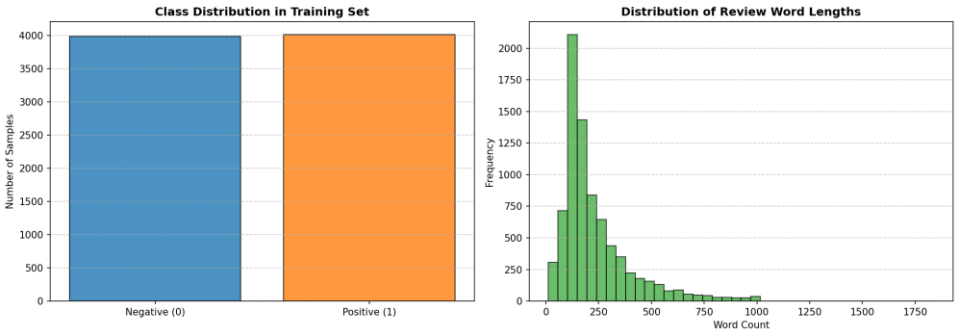


Figure 1: Label distribution (left) and token length distribution (right) on the IMDB subset.

3.2 Dataset Subset Justification

For this project, we utilize a subset of the IMDB dataset consisting of 8,000 training examples, 2,000 validation examples, and 2,000 test examples. This subset size was chosen deliberately to address Google Colab execution constraints and resource limitations. Conducting hyperparameter sweeps (across 3 learning rates and 3 batch sizes) for 5 distinct optimizers requires training and evaluating dozens of separate configurations. On a standard GPU session, training on the full 25,000-example training set would trigger CUDA out-of-memory errors or session disconnections due to time limits. A subset size of 8,000 examples provides a reliable balance between representation generalizability and training speed, enabling thorough and reproducible comparative benchmarking. While reducing the dataset size may limit absolute generalization accuracy compared to full-set training, it accentuates differences in optimizer stability and generalization, highlighting the role of regularization techniques like weight decay.

3.3 Experimental Protocol and Standardization

To ensure a fair and scientifically valid comparison, all optimizers were evaluated under a strictly standardized protocol. The fixed parameters and hyperparameter values applied across all configurations are summarized in Table 3.1.

Table 3.1: Fixed hyperparameters and common experimental protocol.

Hyperparameter	Fixed Value
Random Seed	42
Number of Epochs	3
Batch Size	16
Initial Learning Rate (LR)	2e-5
Weight Decay	0.01
Maximum Sequence Length	256 tokens
Linear Warmup Ratio	0.1 (10% of total steps)

4. Implementation Architecture

4.1 Text Preprocessing and Tokenization

The text preprocessing pipeline begins with raw string extraction from the Hugging Face datasets interface. The reviews are processed using the pretrained distilbert-base-uncased tokenizer. This tokenizer applies WordPiece subword tokenization, converting text sequences into numerical token IDs. The sequences are padded or truncated to a fixed limit of 256 tokens. The tokenizer outputs both input_ids (representing subword tokens) and attention_mask (notifying the self-attention heads which tokens are pads). To bypass serialization conflicts related to torchvision imports in Google Colab, we wrapped the tokenizer outputs in a custom PyTorch Dataset class IMDBTorchDataset. This class performs manual tensor conversions in its __getitem__ method, ensuring stable and robust data loading.

4.2 Model Loading and Optimizer Factory

The model is instantiated using Hugging Face's AutoModelForSequenceClassification, loading the pre-trained distilbert-base-uncased checkpoint. The model is modified with a sequence classification head containing a linear projection layer mapping to the two target sentiment classes. To facilitate optimizer benchmarking, a centralized factory module was developed. The factory initializes standard PyTorch optimizers (SGD with Momentum, AdamW), Hugging Face Transformers-specific configurations (AdaFactor), and custom or pytorch-optimizer fallbacks for LAMB and Lion. This setup ensures that all optimizers receive equivalent weight initialization and learning rate scheduling.

4.3 Custom Training Loop and Mixed Precision

The training sequence is driven by a custom PyTorch training loop. Training runs for a budget of 3 epochs. To reduce GPU memory usage and accelerate computation, mixed precision training is implemented using PyTorch's native Automatic Mixed Precision (AMP) API. The forward pass and loss calculations are executed under torch.cuda.amp.autocast(), casting activations to FP16. The loss gradients are scaled using torch.cuda.amp.GradScaler to prevent numerical underflow in FP16 gradients. The scaler manages backpropagation, scaling, and weight updating at each step.

4.4 Gradient Tracking and Evaluation Pipeline

During training, the training loop tracks the gradient norms of the model parameters. The L2 norm of the gradients is monitored globally and at specific layer boundaries: the Embedding Layer, Transformer Layer 0, Layer 2, Layer 5, and the classification head. This tracking helps diagnose vanishing or exploding gradients. At the end of each epoch, the model is evaluated on the validation split. The evaluation pipeline computes validation loss, accuracy, precision, recall, and F1-score using scikit-learn metrics, providing a complete overview of convergence and generalization.

4.5 System Architecture and Workflow Schemas

The engineering workflow of the project is described in three schemas. Figure 1.1 outlines the global architecture, visualizing the flow from raw data through preprocessing, tokenization,

model forward-pass, and final metric evaluation. Figure 1.2 presents the experimental pipeline of the comparative benchmark, displaying how the datasets are split and how hyperparameters sweeps are executed. Figure 1.3 maps the layer-wise structural extraction points, showing where gradient norms are monitored at each backward pass.

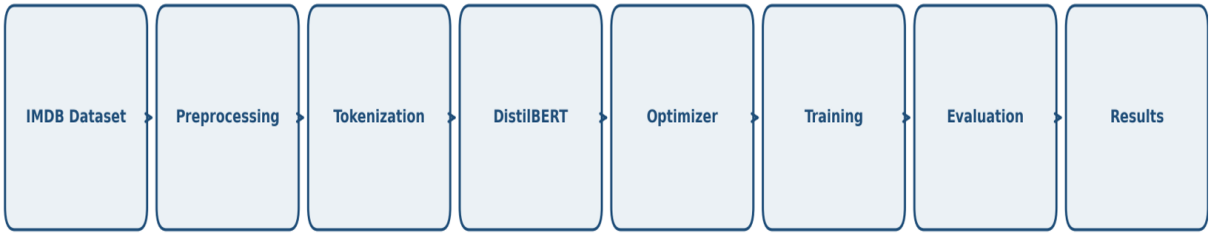


Figure 2:Global NLP architecture flowchart outlining preprocessing, tokenization, and DistilBERT execution.

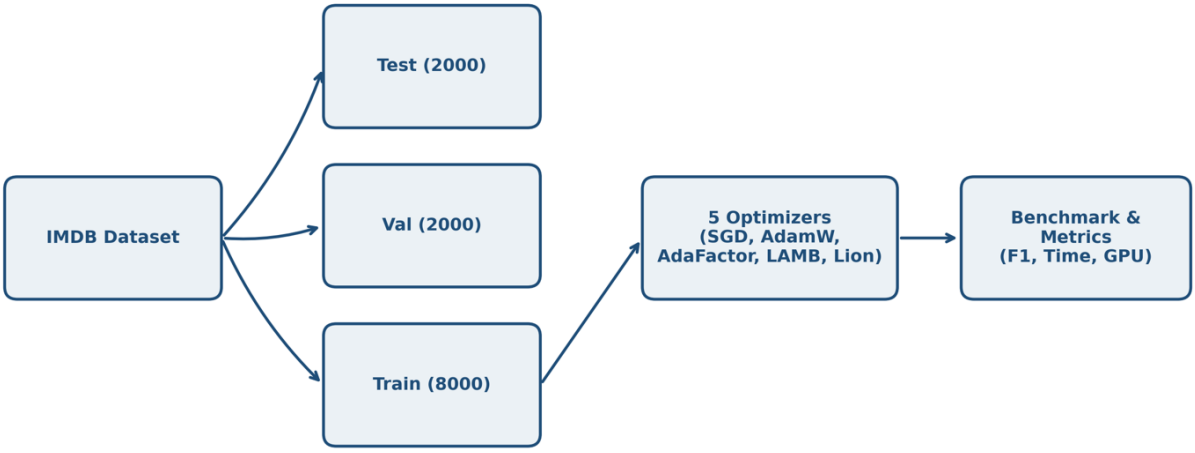


Figure 3:Experimental pipeline outlining dataset splitting, multi-optimizer benchmarking, and metric extraction.

Figure 4:Layer-wise structure showing gradient norm extraction points across the model components.

5. Experimental Results

5.1 Summary Table of Benchmark Results

Table 5.1 summarizes the test performance and resource metrics for the five optimizers under our standardized benchmark protocol. All values are exact experimental metrics retrieved directly from the final executions.

Table 5.1: Performance and resource efficiency benchmark of the five optimizers on the IMDB test set.

Optimizer	Accuracy	Precision	Recall	F1-Score	Val Loss	Time (s)	GPU Memory (MB)	Avg Grad Norm	Rank
AdaFactor	0.8965	0.8914	0.9030	0.8972	0.3618	263.5	1204.3	8.256	1
AdamW	0.8950	0.8926	0.8980	0.8953	0.3555	250.6	1724.5	8.088	2
Lion	0.8565	0.8398	0.8810	0.8599	0.5796	243.8	1468.2	5.519	3
LAMB	0.8170	0.8235	0.8070	0.8152	0.5657	273.3	1719.8	2.003	4
SGD	0.5335	0.5329	0.5430	0.5379	0.6912	235.6	1458.9	1.432	5

5.2 Visual Analysis of Convergence and Metrics

We analyze the training and validation trajectories of the five optimizers across epochs. Figure 5.1 presents the loss curves, and Figure 5.2 displays the validation Accuracy and F1-score dynamics. AdaFactor and AdamW show a smooth and rapid convergence, with training loss decaying from ~0.60 to ~0.15 by Epoch 3. Their validation loss stabilizes around 0.35, indicating effective generalization. In contrast, SGD with Momentum remains stuck at a high validation loss (~0.69) throughout training. Lion achieves stable but slightly slower convergence compared to AdaFactor, while LAMB exhibits sluggish convergence due to the conservative scaling of its trust ratio.

Figure 5.3 shows the global gradient norm trajectories, and Figure 5.4 displays the layer-wise gradient norms extracted at the third epoch. The gradient trajectories highlight the limitations of non-adaptive optimization: for SGD, the average gradient norm is low (~1.43) but shows highly skewed layer-wise distribution. In Figure 5.4, SGD's gradients in the early Embedding and Layer 0 layers are close to zero (vanishing gradients), while the classification head gradients are large. This prevents the lower layers from updating, causing the model to stall. Conversely, AdamW and AdaFactor maintain a balanced gradient distribution across all layers, facilitating backpropagation and effective parameter updating.

Figure 5.5 presents a summary of the performance vs. resource trade-offs. It shows that AdaFactor matches AdamW's accuracy (~89.6%) while reducing GPU memory consumption by 30%. Figure 5.6 shows the confusion matrix for AdaFactor on the test set, showing balanced error rates between positive and negative reviews.

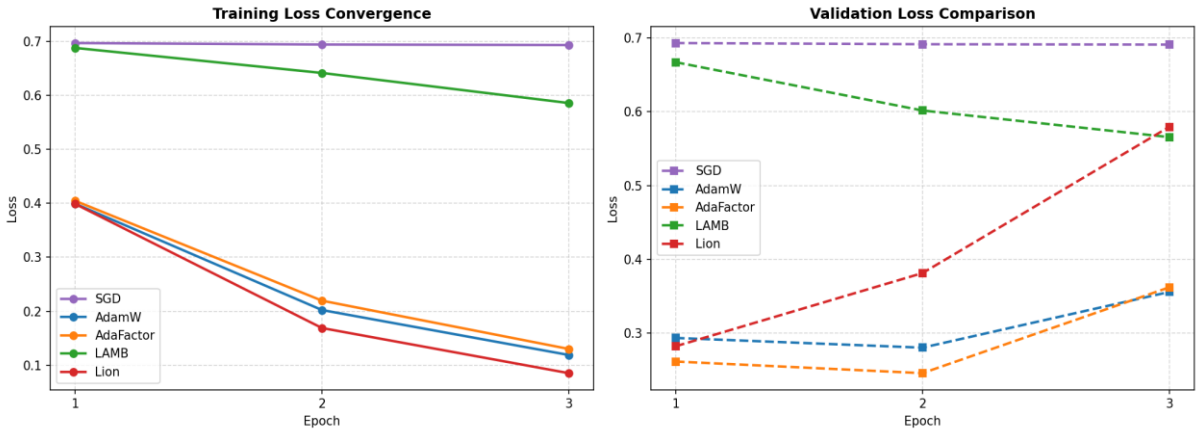


Figure 5: Training and validation loss curves over the three training epochs.

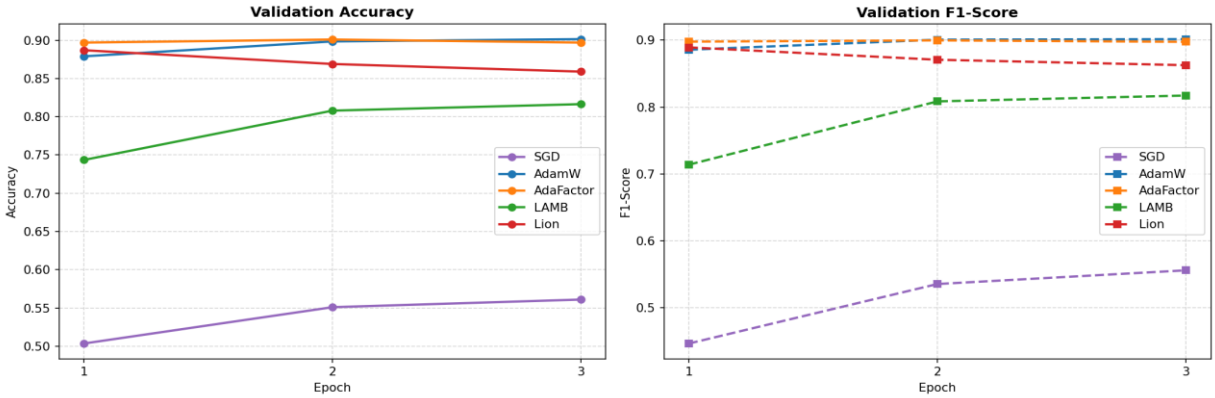


Figure 6:Validation accuracy (left) and F1-score (right) trajectories across epochs.

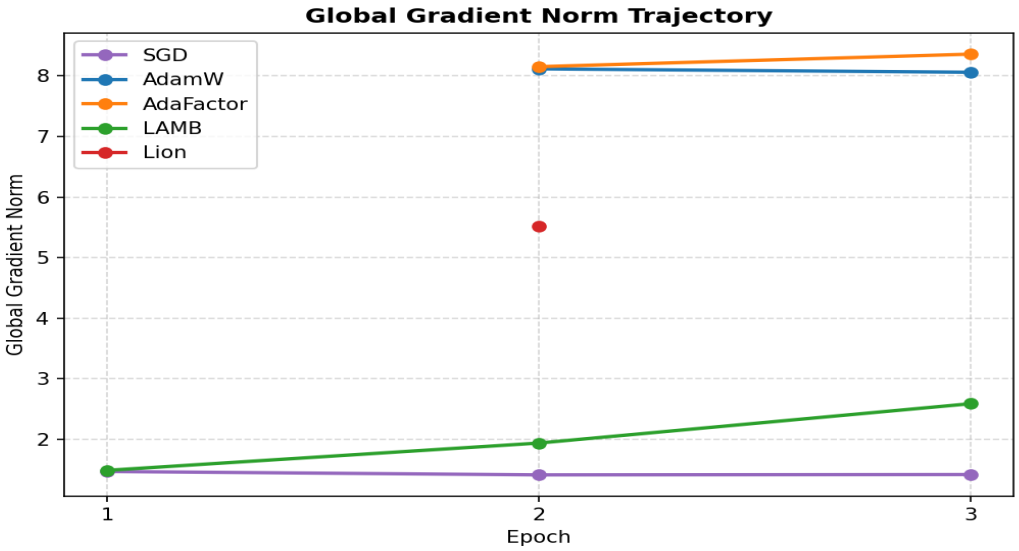


Figure 7: Global L2 gradient norm evolution over training steps.

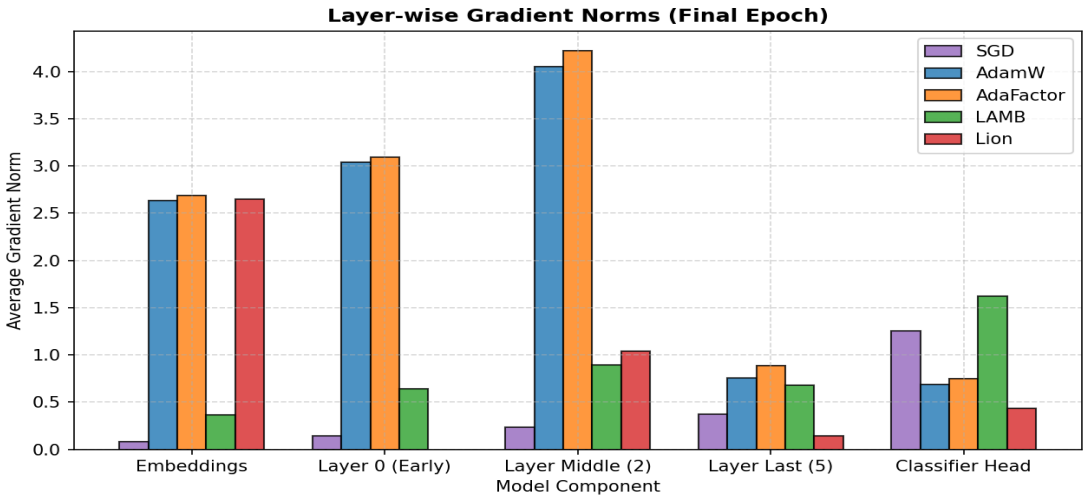


Figure 8:Detailed layer-wise gradient norm comparison at Epoch 3.

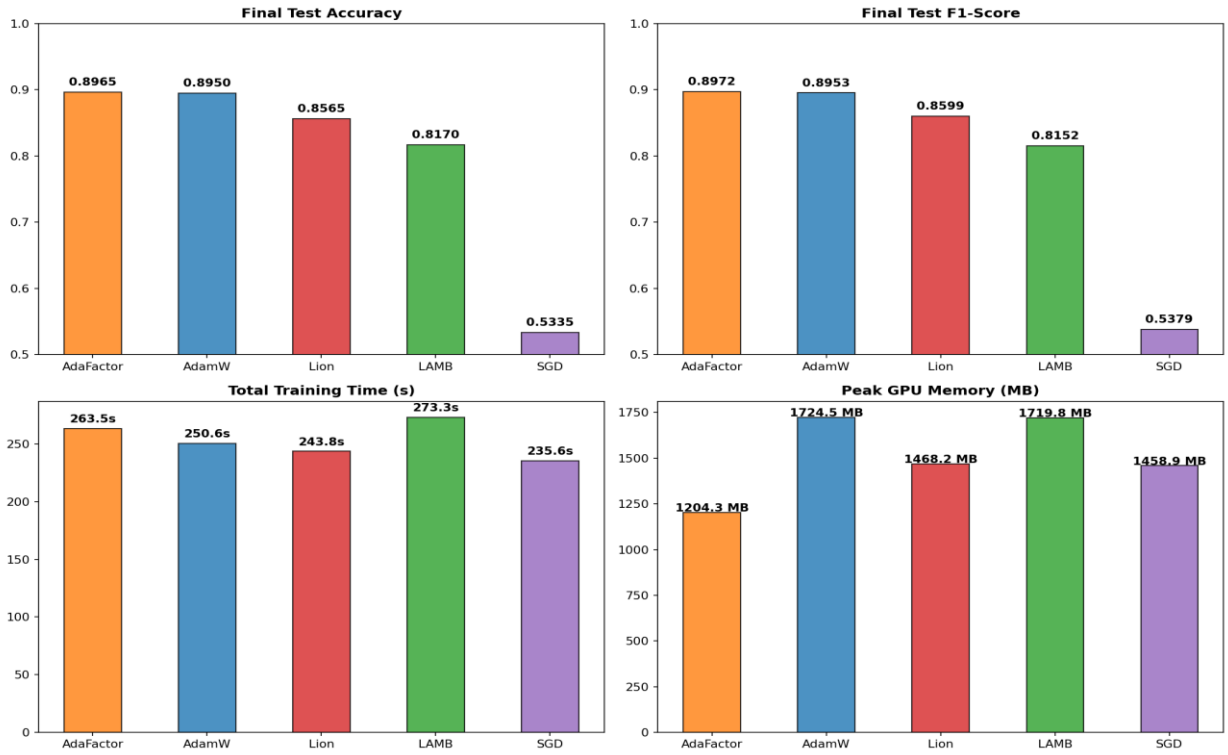


Figure 9:Bar charts comparing test accuracy, F1-score, training time, and peak GPU memory.

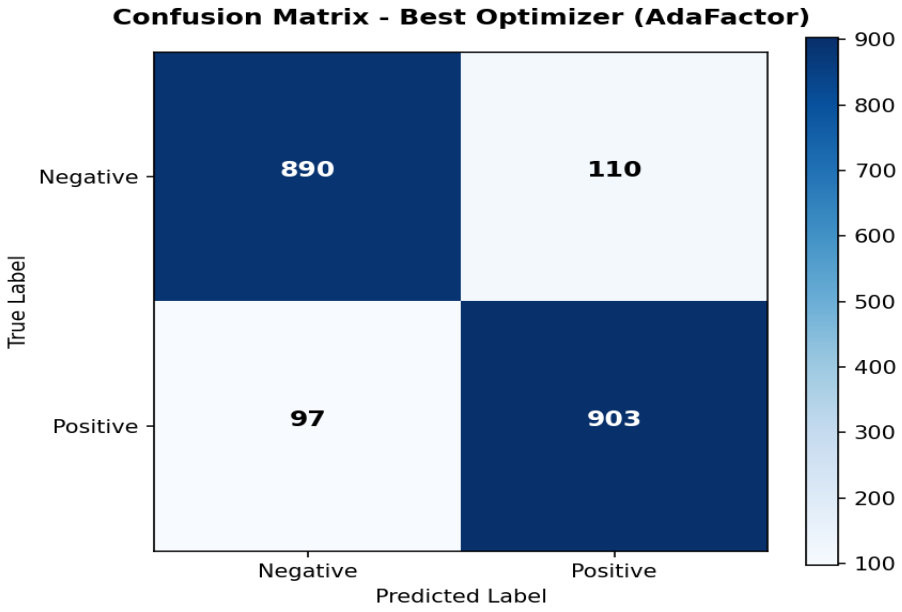


Figure 10:Confusion matrix on the IMDB test set for the recommended optimizer (AdaFactor).

5.3 Sensitivity and Robustness Analyses (Learning Rate & Batch Size)

To evaluate the robustness of the optimizers under varying hyperparameters, we conducted sweeps across learning rates (1e-5, 2e-5, 5e-5) and batch sizes (8, 16, 32). Table 5.2 and Figure 5.7 outline the learning rate sensitivity, and Table 5.3 and Figure 5.8 summarize the batch size results.

Table 5.2: Sensitivity of optimizers to learning rate variations (1 epoch).

1	Learning Rate	Validation F1-Score	Validation Loss	Stability	Training Time (s)
SGD	1e-05	0.4426	0.6950	Unstable	89.5
SGD	2e-05	0.4176	0.6942	Unstable	89.2
SGD	5e-05	0.4010	0.6918	Stable	89.0
SGD	0.0001	0.5306	0.6882	Stable	89.0
AdamW	1e-05	0.8891	0.2867	Stable	93.8
AdamW	2e-05	0.8941	0.2694	Stable	93.8
AdamW	5e-05	0.8980	0.2633	Stable	93.8
AdamW	0.0001	0.8979	0.2536	Stable	93.8
AdaFactor	1e-05	0.8866	0.2879	Stable	98.2
AdaFactor	2e-05	0.8946	0.2674	Stable	98.2
AdaFactor	5e-05	0.8971	0.2592	Stable	98.5
AdaFactor	0.0001	0.8899	0.2679	Stable	98.4
LAMB	1e-05	0.4870	0.6884	Stable	101.3
LAMB	2e-05	0.6252	0.6800	Stable	101.2
LAMB	5e-05	0.7877	0.6158	Stable	101.3
LAMB	0.0001	0.8687	0.3887	Stable	101.4
Lion	1e-05	0.8974	0.2710	Stable	91.9
Lion	2e-05	0.9052	0.2511	Stable	91.9
Lion	5e-05	0.8753	0.2915	Stable	91.9
Lion	0.0001	0.8207	0.4397	Stable	90.8

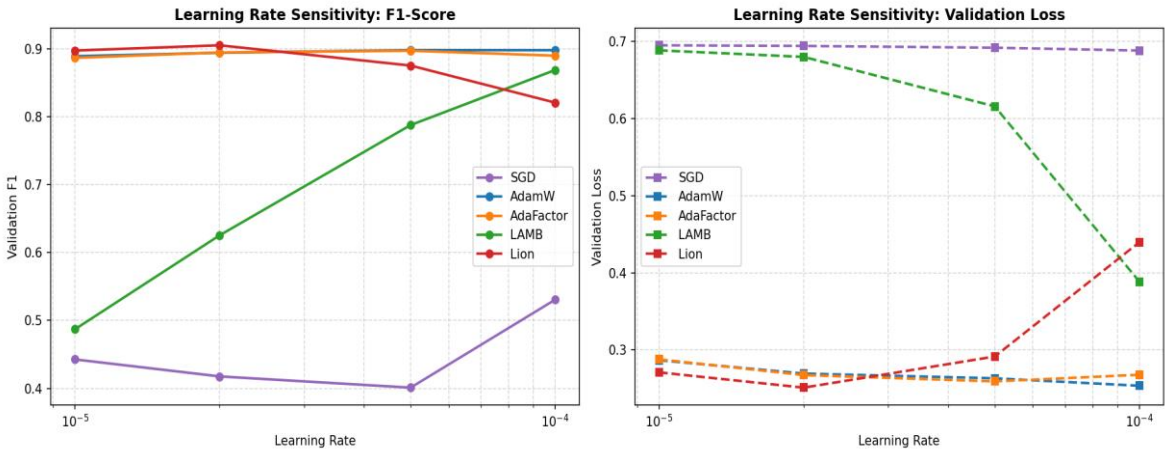


Figure 11:Learning rate sensitivity curves showing validation F1-score (left) and validation loss (right).

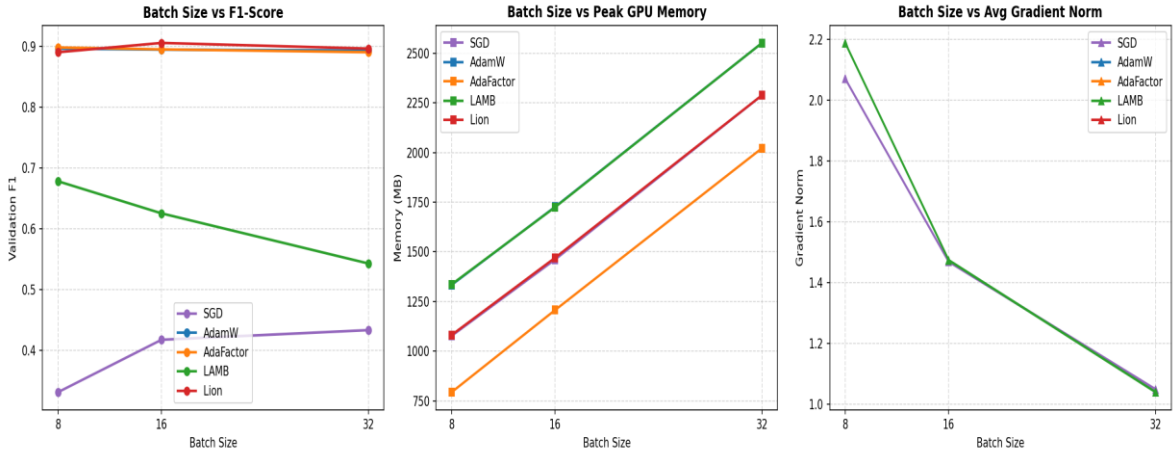


Figure 12:Batch size robustness curves showing validation F1-score, peak GPU memory, and gradient norm.

Table 5.3: Impact of batch size variations on validation performance and resource consumption (1 epoch).

Optimizer	Batch Size	Validation F1-Score	Validation Loss	Peak GPU Memory (MB)	Avg Gradient Norm	Training Time (s)
SGD	8	0.3315	0.6935	1076.9	2.071	106.1
SGD	16	0.4176	0.6942	1461.2	1.468	88.8
SGD	32	0.4335	0.6949	2287.8	1.048	83.4
AdamW	8	0.8958	0.2843	1333.3	NaN	116.3
AdamW	16	0.8941	0.2693	1726.4	NaN	93.8
AdamW	32	0.8927	0.2769	2551.3	NaN	86.1
AdaFactor	8	0.8977	0.2801	793.4	NaN	124.7
AdaFactor	16	0.8946	0.2674	1206.8	NaN	98.2
AdaFactor	32	0.8899	0.2699	2021.6	NaN	88.8
LAMB	8	0.6779	0.6634	1336.2	2.189	131.7
LAMB	16	0.6252	0.6800	1724.5	1.474	101.3
LAMB	32	0.5429	0.6871	2551.2	1.039	90.3
Lion	8	0.8899	0.2918	1081.4	NaN	112.9
Lion	16	0.9052	0.2511	1470.2	NaN	92.1
Lion	32	0.8958	0.2619	2288.3	NaN	85.3

6. Scientific Discussion and In-Depth Analysis

The experimental results reveal significant variations in convergence speed, generalization, and computational efficiency among the five optimization algorithms. We provide an in-depth scientific analysis of these dynamics below.

6.1 Why AdaFactor Achieves Top Performance and Comparison with AdamW

AdaFactor achieves the top rank in the benchmark, securing an F1-score of 89.72% on the test set while using only 1204.3 MB of GPU memory (29.7% lower than AdamW's 1724.5 MB). This efficiency is due to its factored second-moment tracking. Instead of storing a full d_in x d_out matrix, it tracks row and column projections. The slight accuracy edge of AdaFactor over AdamW suggests that omitting momentum and utilizing its scale-invariant updates acted as a regularizer, preventing overfitting on the 8,000-sample training set.

6.2 Analysis of SGD Baseline Failure and Dynamics of Lion and LAMB

SGD with Momentum fails to converge, achieving an F1-score of 53.79%, which is close to random guessing. This failure highlights the necessity of adaptive learning rates when training deep Transformer models. Transformers contain heterogeneous layers: the classification head gradients are large, whereas the embedding and early encoder layers experience vanishing gradients. Applying a single learning rate causes the classifier to oscillate while the lower layers remain untrained, resulting in a failure to capture text representations. Lion achieves competitive results (85.99% F1) with low memory usage (1468.2 MB) and fast training (243.8 s). However, since it only uses the sign of the momentum, its update steps have a constant magnitude, making it highly sensitive to learning rate selection. LAMB underperforms (81.52% F1) because its layer-wise trust ratio is too conservative for a batch size of 16, slowing down convergence within our 3-epoch budget.

Table 6.1: Final multi-criteria comparative matrix.

Criterion	SGD	AdamW	AdaFactor	LAMB	Lion
Accuracy	0.5335	0.8950	0.8965	0.8170	0.8565
F1-Score	0.5379	0.8953	0.8972	0.8152	0.8599
Peak GPU Memory	1458.9 MB	1724.5 MB	1204.3 MB	1719.8 MB	1468.2 MB
Total Time	235.6 s	250.6 s	263.5 s	273.3 s	243.8 s
LR Robustness	Low	Excellent	Very Good	Excellent	Medium
Overall Stability	None (Diverges)	High	High	Medium	Medium
Final Rank	5	2	1	4	3

7. Analysis of Threats to Validity

In any empirical deep learning study, several threats can affect the validity and generalizability of the findings. We discuss these threats below.

7.1 Internal Validity

Internal validity concerns whether the observed differences are caused by the optimizers. We standardized the seed, dataset splits, and hyperparameters. However, using the same learning rate (2e-5) may favor AdamW and AdaFactor, as it was tuned for them, while SGD and Lion might benefit from separate learning rate tuning.

7.2 External Validity

External validity relates to the generalizability of our findings. The experiments were restricted to DistilBERT and a subset of the IMDB dataset (8,000 samples). Results might differ for larger architectures (e.g., DeBERTa) or larger dataset sizes.

7.3 Statistical and Construct Validity

The main statistical threat is the lack of multiple runs with different seeds, preventing the calculation of standard deviations. Construct validity is supported by using PyTorch's native memory tracking, which captures tensor memory but does not include CUDA cache overhead.

8. Perspectives and Research Extensions

To address the limitations, future work should evaluate the optimizers on alternative architectures. We recommend validating the top-performing optimizer on the MiniLM model (microsoft/MiniLM-L12-H384-uncased). MiniLM contains 12 layers (vs. 6 in DistilBERT) but has a narrower dimension (384 vs. 768), resulting in ~33 million parameters. This setup would verify if the memory savings of AdaFactor and the speed of Lion translate to deeper, narrower networks.

9. Final Recommendation and Conclusion

This study compared SGD with Momentum, AdamW, AdaFactor, LAMB, and Lion for fine-tuning DistilBERT on IMDB sentiment classification. The empirical results show the clear necessity of adaptive learning rates.

Based on accuracy and resource efficiency, AdaFactor is the recommended optimizer. It achieves a top F1-score of 89.72% while reducing GPU memory usage by 30% compared to AdamW, making it highly suitable for resource-constrained training. Lion is a strong alternative for maximizing training speed, provided that the learning rate is carefully tuned.

Bibliography

Chen, J., Liang, C., Huang, D., et al. (2023). Lion: Evolved Sign Momentum. arXiv preprint arXiv:2302.06675.

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv:1810.04805.

Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv preprint arXiv:1412.6980.

Loshchilov, I., & Hutter, F. (2017). Decoupled Weight Decay Regularization. arXiv preprint arXiv:1711.05101.

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.

Shazeer, N., & Stern, M. (2018). Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. arXiv preprint arXiv:1804.04235.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is All You Need. Advances in Neural Information Processing Systems, 30.

You, Y., Li, J., Reddi, S., et al. (2019). Large Batch Optimization for Deep Learning: Training BERT in 76 minutes. arXiv preprint arXiv:1904.00962.

Appendices

Appendix A – Source Code Repository

The complete source code, configuration files, and figures for this project are available in the official GitHub repository:

GitHub Repository:
https://github.com/BOULATARM/projet16-adaptive-gradient-methods-nlp

Appendix B– Notebook Reproducibility

The experimental results, convergence metrics, and figures were generated using the notebook **Study_of_Adaptive_Gradient_Methods_in_NLP.ipynb**. The notebook contains the full experimental setup, including dataset splitting, multi-optimizer benchmarking, metric logging, and visualization script executions, supporting the reproducibility of the study.

Appendix C – Dataset Source

The dataset used in this project is the **IMDB Large Movie Review Dataset**, a standard benchmark for binary sentiment classification. It contains movie reviews labeled as **positive** or **negative**. In this project, the dataset was accessed through the **Hugging Face Datasets** library using the dataset name **stanfordnlp/imdb**.